Software Engineering

Introduction to Unit Testing

What we have learned...

- Module Resolution and System
- Enums
- Typescript Best Practices

What we will learn...

- Overview of unit testing
- Importance of unit testing
- Setup testing in TS
- Testing TS with Jest
- Jest matchers

✋ Any first questions?

What is Testing?

Software testing is the process of evaluating and verifying that a software product or application does what it’s **supposed to do**.

Why Software Testing Important?

Cyberpunk 2077

Remember Cyberpunk 2077 release?

The game came out with so many bugs and glitches because it is not properly tested.

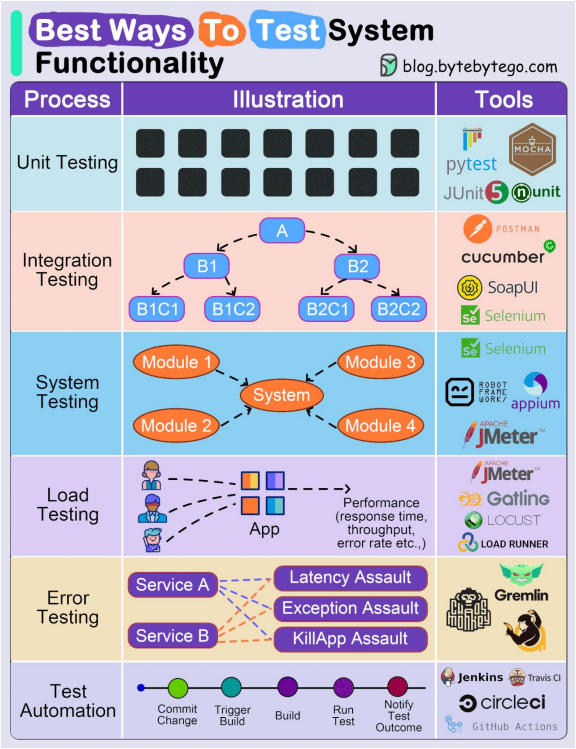
Why Software Testing Important?

Boeing 737 max

Remember Boeing 737 Max crash?

The plane crash due to poorly tested software because Boeing's 737 Max Software Outsourced to $9-an-Hour Engineers

Type of Software Testing



What is Unit Testing?

Unit Testing Introduction

Unit Testing:

Ensures individual code components work **correctly in isolation**.

What are the benefits of unit testing?

- **Efficient bug discovery**

If there are any input, output, or logic-based errors within a code block, your unit tests help you catch them before the bugs reach production. When code changes, you run the same set of unit tests—alongside other tests such as integration tests—and expect the same results. If tests fail (also called broken tests) it indicates regression-based bugs.

Unit testing also helps finds bugs faster in code. Your developers don't spend a large amount of time on debugging activities. They can quickly pinpoint the exact part of the code that has an error.

What are the benefits of unit testing?

- **Documentation**

It's important to document code to know exactly what that code is supposed to be doing. That said, unit tests also act as a form of documentation.

Other developers read the tests to see what behaviors the code is expected to exhibit when it runs. They use the information to modify or refactor the code. Refactoring code makes it more performant and well-composed. You can run the unit testing again to check that code works as expected after changes.

How do developers use unit tests?

Developers leverage unit tests at various stages of the software development lifecycle,

often following **Test-Driven Development** (TDD) and **Behavior-Driven Development** (BDD) practices:

How do developers use unit tests?

- **Test-Driven Development (TDD):**

TDD is a development approach where developers write tests to define the desired behavior of a piece of software before writing the actual code.

By following TDD, developers create unit tests first, based on the functional requirements, and then implement the code to pass those tests.

TDD ensures that the code meets the specified requirements, improves code quality, and provides a safety net for refactoring.

Test-Driven Development (TDD):

```
// calculator.test.js
const Calculator = require('./calculator');

describe('Calculator', () => {
  test('add method should return the sum of two numbers', () => {
    const calc = new Calculator();
    expect(calc.add(2, 3)).toBe(5);
  });
});
```

```
npx jest
```

Test-Driven Development (TDD):

```
// calculator.js
class Calculator {
  add(a, b) {
    return a + b;
  }
}

module.exports = Calculator;
```

```
npx jest
```

How do developers use unit tests?

- **Behavior-Driven Development (BDD):**

BDD is an agile software development methodology that focuses on defining the behavior or functionality of the software from the end user's perspective.

In BDD, tests are written in a human-readable format using a Given-When-Then structure to describe the expected behavior. BDD tests verify the behavior of the software as a whole, covering multiple units of code working together to deliver specific features or functionalities.

Behavior-Driven Development (BDD):

Feature: User Login

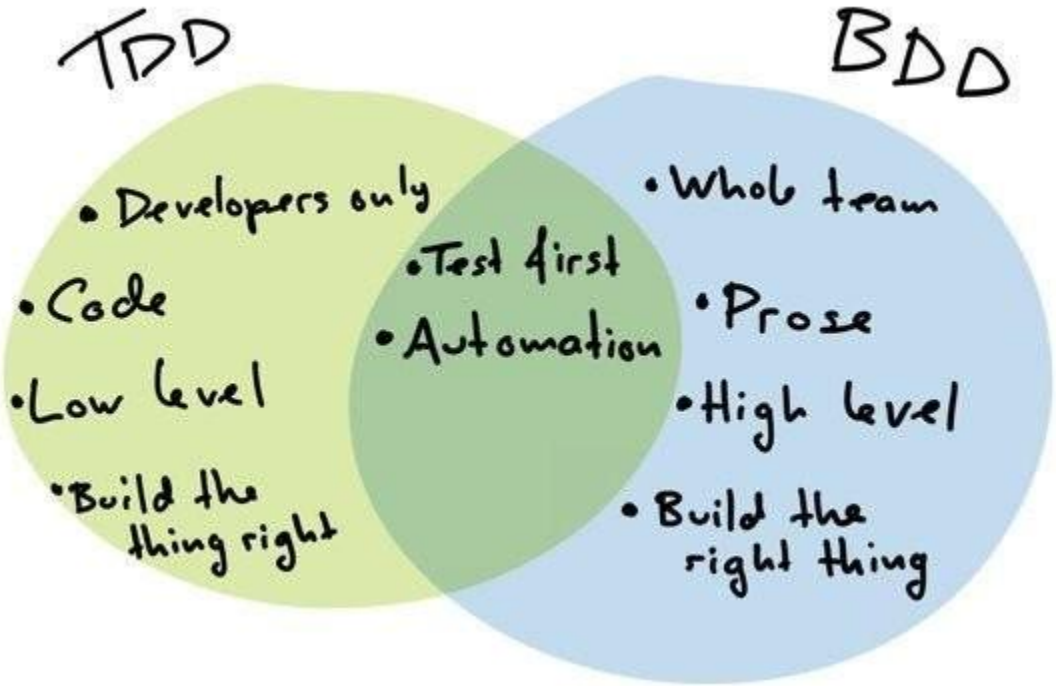
Scenario: Successful login with valid credentials

- **Given** a user with the username "johndoe" and password "password123" exists
- **When** the user attempts to log in with the username "johndoe" and password "password123"
- **Then** the user should be redirected to the dashboard

Scenario: Failed login with invalid credentials

- **Given** a user with the username "johndoe" exists
- **When** the user attempts to log in with the username "johndoe" and password "wrongpassword"
- **Then** the user should see an error message "Invalid username or password"

How do developers use unit tests?



When is unit testing less beneficial?

Unit testing isn’t always required for every single test case in every single block of code in every single project. Here are some examples of when unit testing could potentially be omitted.

- **When time is constrained**

Even with generative unit testing frameworks, writing new unit tests takes a significant amount of your developers' time. While input and output-based unit tests may be easy to generate, logic-based checks are more difficult.

Once your developers start writing tests, they also see refactoring opportunities in the block of code and get distracted from completing them. This can lead to extended development timelines and budget issues.

When is unit testing less beneficial?

- **UI/UX applications**

When the main system is concerned with look and feel rather than logic, there may not be many unit tests to run. Other types of testing, such as manual testing, are a better strategy than unit testing in these cases.

- **Legacy codebases**

Writing tests to wrap around existing legacy code can prove to be near impossible, depending on the style of the written code. Because unit tests require dummy data, it can also be too time-intensive to write unit tests for highly interconnected systems with a lot of data parsing.

- **Rapidly evolving requirements**

Depending on the project, the software can grow, change directions, or have whole parts scrapped altogether in any given work sprint. If requirements are likely to change often, there's not much reason to write unit tests each time a block of code is developed.

Unit testing strategies

To create unit tests, you can follow some basic techniques to ensure coverage of all test cases.

- **Logic checks**

Does the system perform the right calculations and follow the right path through the code given a correct, expected input? Are all paths through the code covered by the given inputs?

- **Boundary checks**

For the given inputs, how does the system respond? How does it respond to typical inputs, edge cases, or invalid inputs?

Let's say you expect an integer input between 3 and 7. How does the system respond when you use a 5 (typical input), a 3 (edge case), or a 9 (invalid input)?

Unit testing strategies

- **Error handling**

When there are errors in inputs, how does the system respond? Is the user prompted for another input? Does the software crash?

- **Object-oriented checks**

If the state of any persistent objects is changed by running the code, is the object updated correctly?

Unit Testing Key Aspect

- **Isolation**:

Unit tests are designed to be isolated, meaning they focus on testing a specific piece of functionality in isolation from the rest of the codebase. This is achieved by using mock objects or stubs to simulate the behavior of external dependencies.

- **Automated**:

Unit tests are typically automated, which means they can be executed automatically by a testing framework without manual intervention. This allows for quick and repeatable testing.

- **Fast Execution**:

Unit tests are meant to be fast to execute. They should complete in a matter of milliseconds or seconds, allowing for rapid feedback during development.

- **Repeatable**:

Unit tests should produce consistent results each time they are run. This means that if the code hasn't changed, the tests should always pass.

Testing with Jest

Testing Node Applications with Jest

Here are some key features and characteristics of Jest:

Zero Configuration: Jest aims to provide a seamless testing experience with minimal setup. It comes preconfigured with sensible defaults, so you can start writing tests right away.

Built-in Test Runner: Jest provides a built-in test runner that executes your tests and provides real-time feedback on their progress. It can run tests in parallel, optimizing the testing process.

Automatic Mocking: Jest automatically mocks dependencies, allowing you to focus on testing the specific behavior of your code without worrying about its dependencies.

Snapshot Testing: Jest includes a powerful feature called snapshot testing. It allows you to capture a "snapshot" of your component or data structure and compare it against future snapshots to detect unintended changes.

Matchers: Jest provides a wide range of built-in matchers for making assertions in your tests. These include common assertions like expect(x).toBe(y) and more complex ones for handling different data types.

Testing Node Applications with Jest

Mock Functions: Jest provides a way to create mock functions that you can use to spy on function calls and control their behavior in tests.

Code Coverage: Jest can generate code coverage reports, showing you which parts of your code are covered by tests and which are not. This can help you identify areas that may need additional testing.

Test Suites and Test Cases: Jest organizes tests into suites and test cases, making it easy to group related tests and run them selectively.

Support for Various Testing Frameworks: While Jest is commonly used for testing React applications, it can be used with other frontend libraries and frameworks as well as backend Node.js code.

Extensibility: Jest is highly extensible, allowing you to customize its behavior with plugins and configurations.

What is Jest?

Jest is a delightful JavaScript testing framework with a focus on simplicity.

It works with projects using Babel, TypeScript, Node.js, React, Angular, Vue.js, and more. Jest provides a rich set of features for writing and managing tests, including assertions, mocking, and code coverage reporting.

Setting Up Jest

You can follow from Github Repository examples

- Basically you just need setup TypeScript project as usual
- Install Jest
  - **npm install jest @types/jest ts-jest --save-dev**
- Update tsconfig.json to include Jest configuration for TypeScript
- Add Jest configuration in package.json
- Run the tests:

Writing You First Test

In Jest, tests are written using the test function. Each test typically follows this pattern:

```
test('description of the test', () => {
  // Test logic goes here
});
```

Inside the test function, you write the logic to exercise the code you want to test and use Jest's assertion functions to verify the expected behavior.

Jest - Assertions

Jest provides a variety of assertion functions to check for different conditions. Some common ones include:

```
expect(value).toBe(expected)
// Asserts that value is equal to expected.

expect(value).toEqual(expected)
// Asserts that value is deeply equal to expected.

expect(value).not.toBe(expected)
//Asserts that value is not equal to expected.
```

Jest - Concept

1. Test:

A test is a piece of code that verifies whether a specific portion of your program behaves as expected under certain conditions. Tests are usually written to cover different functionalities of your application, ensuring that each part works correctly.

2. Test Suite:

A test suite is a collection of test cases or test files that focus on testing a specific module, component, or functionality of your application. Test suites help organize your tests and make it easier to run them together.

3. Pass/Fail:

When you run tests, each test case will either pass or fail. A test passes if the actual outcome matches the expected outcome defined in the test assertion. Conversely, a test fails if the actual outcome differs from the expected outcome, indicating a problem in your code.

Jest - Lifecycle

beforeAll:

The beforeAll function is called once before any of the test cases (i.e., test functions) in a test suite are executed. It's typically used to set up common resources or perform initializations that are required by all tests within the suite.

```
beforeAll(() => {
  // Perform setup operations here, such as initializing a database connection
  initializeDatabase();
});
```

Jest - Lifecycle

afterAll:

Conversely, the afterAll function is called once after all the test cases in a test suite have finished running. It's commonly used to clean up resources or perform finalizations after all tests have completed.

```
afterAll(() => {
  // Perform teardown operations here, such as closing a database connection
  closeDatabaseConnection();
});
```

Practice & Demo

https://github.com/retzd-tech/html-dom-ts-testing.git

QnA & Discussion

Today’s Recap

Thank you!

Being objective is a good way to be a solutive.
We are not coder, We are Problem Solver.

Share your learning journey with us!
Let's inspire each other with your stories

@revou_id RevoU revoudotco